

**LAPORAN PROYEK PEMROGRAMAN DASAR GAME SUDOKU
SEDERHANA DENGAN LIBRARY PYGAME**



PENYUSUN :

Bima Setia Sugiharto (24031554040)

Rafael Hartono (24031554166)

Indah Faizah Salsabillah R. W. (24031554000)

KELOMPOK 9

KELAS 2024F

DOSEN PENGAMPU : HARMON PRAYOGI, M. SC.

PROGRAM STUDI SAINS DATA

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

UNIVERSITAS NEGERI SURABAYA

2024

DAFTAR ISI

DAFTAR ISI	1
BAB I.....	2
PENDAHULUAN	2
1. Latar Belakang	2
2. Rumusan Masalah	2
3. Tujuan.....	2
BAB II	3
ANALISIS DAN PERANCANGAN	3
1. Diagram Alur.....	3
2. Spesifikasi Program.....	4
3. Sketsa Desain Antarmuka	5
BAB III.....	9
IMPLEMENTASI.....	9
1. Import.....	9
2. Class SudokuGame.....	10
3. Inisialisasi (__init__).....	10
4. Metode-Metode Utama	11
5. Fitur-Fitur Game Sudoku By IRAMA.....	17
LAMPIRAN KODE UTAMA.....	19
DAFTAR PUSTAKA	41

BAB I

PENDAHULUAN

1. Latar Belakang

Python merupakan salah satu bahasa pemrograman yang banyak digunakan oleh perusahaan besar maupun para developer untuk mengembangkan berbagai macam aplikasi berbasis desktop, web dan mobile. Python juga banyak dipelajari dikalangan mahasiswa dan kampus, terutama kampus berbasis IT. Pembelajaran pemrograman tidak mudah, namun tidak juga sulit, tergantung bagaimana cara belajarnya, dan seberapa cepat dalam mempelajarinya.(Rahman et al., n.d.)

Teknologi komputer saat ini sudah berkembang sangat cepat, baik dari sisi hardware ataupun dari sisi software. Dari sisi Software, berbagai aplikasi kini dapat diciptakan dengan mudahnya, karena perkembangan berbagai jenis software seiring dengan perkembangan Kecerdasan Buatan. Banyaknya aplikasi-aplikasi yang diciptakan dengan kecerdasan buatan didalamnya, terutama aplikasi-aplikasi game pada dunia komputer, seperti aplikasi game real strategi, game sport, game yang bersifat petualangan (adventure) dan juga game yang bersifat mengasah logika.(Hadinata et al., n.d.)

Salah satu game logika yang kini mulai banyak diminati oleh banyak kalangan adalah “Sudoku”. Sudoku adalah sebuah permainan teka-teki angka sederhana yang telah dikenal seluruh dunia mulai dari anak-anak sampai orang tua. Untuk memainkan puzzle angka ini pemain tidak perlu berpikir keras, tidak harus mempunyai intelektualitas yang tinggi, dan tidak perlu handal berhitung, karena yang dibutuhkan hanya kemampuan berpikir secara logika, kesabaran tinggi dan ketajaman akurasi. (Hadinata et al., n.d.)

Namun, banyak programmer pemula yang masih kesulitan dalam memahami bagaimana proses coding menggunakan Bahasa pemrograman python dan implementasi berbagai konsep dalam pemrograman seperti penggunaan kelas (class) dan alur kerja game sudoku. Oleh karena itu, proyek ini bertujuan untuk membantu mahasiswa memahami lebih jauh konsep dasar tersebut dalam konteks pengembangan game Sudoku menggunakan bahasa pemrograman Python serta library pygame.

2. Rumusan Masalah

Dalam project yang dibuat ini dapat disimpulkan rumusan masalah sebagai berikut:

- a. Bagaimana code yang harus dibuat agar dapat membuat game dengan Python?
- b. Bagaimana alur game sudoku tersebut berjalan?
- c. Bagaimana implementasi game sudoku dengan library Pygame?

3. Tujuan

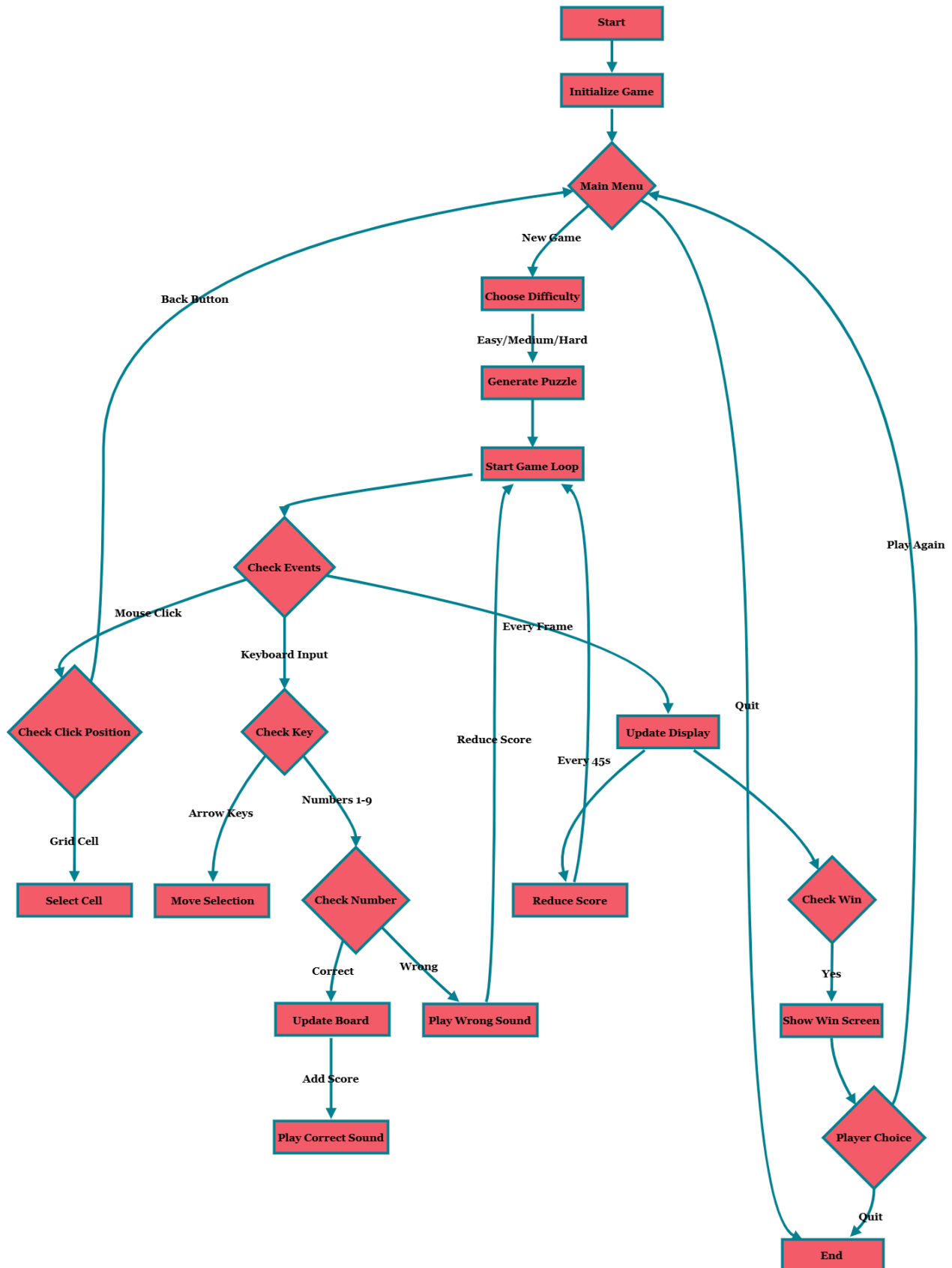
Project ini memiliki beberapa tujuan, yaitu:

- a. Untuk mengetahui code yang harus dibuat agar dapat membuat game dengan Python.
- b. Untuk mengetahui alur game sudoku.
- c. Untuk mengetahui implementasi game sudoku dengan library Pygame.

BAB II

ANALISIS DAN PERANCANGAN

1. Diagram Alur



2. Spesifikasi Program

a) Sistem Operasi:

- **Windows 7/8/10 (32/64 bit)**
- **macOS 10.11 atau lebih baru**
- **Linux dengan desktop environment**

b) Python:

- **Python 3.x (direkomendasikan Python 3.7 atau lebih baru)**
- **pip (Python package installer)**

c) Dependencies/Library:

- **Pygame (pip install pygame)**
- **Pygame mixer untuk audio**

d) Hardware:

- **Processor: 1 GHz**
- **RAM: 2 GB**
- **Storage: 100 MB free space**
- **Display: 800x600 resolution**

e) Input Devices:

- **Keyboard**
- **Mouse**

f) Persyaratan File dan Asset:

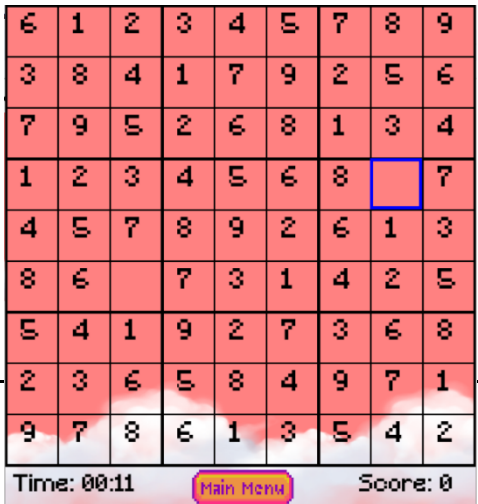
- **File utama Python (.py)**
- **Asset gambar dalam format PNG/JPG**
- **File font (Minecraft.ttf)**
- **File audio (.wav, .mp3)**

g) Aplikasi yang dibutuhkan

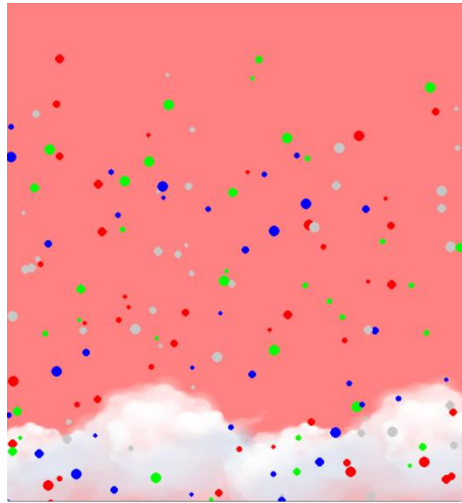
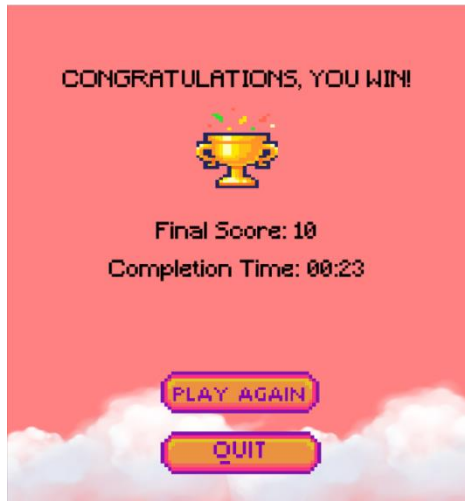
- **Visual Studio Code (VSCODE)**

3. Sketsa Desain Antarmuka

Latar Belakang	Elemen Piala
	
Elemen Tingkat Kesulitan	Elemen Judul
	
Tombol Menu Utama	Tombol Tingkat Kesulitan
 	  
Tombol Kembali ke Menu Utama	Tombol Bermain Kembali
	

Tampilan	Tata Letak
Menu Utama	
Tingkat Kesulitan	
Game	

--	--

Animasi Konfeti	
Tampilan Menang	

BAB III

IMPLEMENTASI

1. Import

a. import pygame

Pygame adalah library utama untuk membuat game sudoku di bahasa pemrograman Python ini. Pygame menyediakan fungsi-fungsi untuk : Menampilkan grafis dan gambar, Mendeteksi input keyboard dan mouse, Mengelola collision detection (mendeteksi tabrakan), Membuat animasi, Menangani event dalam game, Menggambar bentuk geometris (persegi, lingkaran, dll), Menampilkan teks, Mengatur window/layar game

b. from pygame import mixer

Import Mixer adalah modul khusus dari pygame untuk menangani audio yang dimana pada kasus ini berfungsi sebagai pemutaran background audio utama, sound effect jika user meng-click button yang ada di game, sound effect jika user benar menginputkan angka, sound effect jika user salah menginputkan angka, sound effect jika user memenangkan permainan, serta sound effect untuk notifikasi tertentu dalam game.

c. import time

Import time adalah suatu modul untuk mengatur waktu dan delay pada game sudoku ini yang diimplementasikan dengan : Membuat delay/jeda antar aksi, Mengukur waktu eksekusi, Membuat timer/countdown, Mengatur frame rate, Membuat animasi berbasis waktu, Mengatur kecepatan game, Membuat cooldown skill/ability

d. import random

Import random adalah modul untuk menghasilkan nilai acak/random yang dimana pada game ini terdapat 3 tingkat kesulitan, yaitu mudah (easy), menengah (medium), dan sulit (hard). Yang dimana modul random disini berfungsi untuk mengacak tiap angka per tingkat kesulitannya masing masing. Serta membuat algoritma backtracking yaitu teknik pencarian solusi yang dilakukan secara sistematis dan efisien.

e. `import os`

Import os adalah suatu Modul untuk berinteraksi dengan sistem operasi, modul ini lebih digunakan untuk kepentingan kerapian dan efektifitas saja yang dimana pengimplementasiannya yakni, untuk membaca file pada folder tertentu, pada kasus ini membaca folder “assets”, “sounds” yang dimana tempat semua asset atau keperluan game sudoku ini guna memperbagus visual dan pengalaman bermain pengguna.

2. Class `SudokuGame`

Class `Sudoku Game` adalah inti dari permainan Sudoku yang mengelola seluruh mekanisme permainan, mulai dari tampilan grafis hingga logika game. Ini adalah kelas utama yang berfungsi untuk menyatukan elemen-elemen penting dalam game seperti pengaturan grafis, pemrosesan input pengguna, pengelolaan permainan, dan lain-lain.

3. Inisialisasi (`__init__`)



Metode `__init__` berfungsi untuk menginisialisasi semua komponen yang diperlukan dalam game. Salah satunya adalah inisialisasi Pygame dan modul mixer untuk menangani audio. Pygame akan digunakan untuk menangani grafik dan event, sedangkan mixer digunakan untuk memutar efek suara dan musik latar.

Kemudian, di bagian ini juga didefinisikan berbagai warna yang digunakan dalam permainan, seperti putih (WHITE), hitam (BLACK), merah (RED), dan warna-warna lain yang mempercantik tampilan visual. Ukuran window juga diatur dalam variabel `WIDTH` (605 piksel) dan `HEIGHT` (660 piksel) yang akan menampung grid Sudoku serta elemen lainnya.

Selanjutnya, dalam metode ini juga dihitung ukuran setiap kotak dalam grid Sudoku dengan variabel `SQUARE_SIZE`, yang menentukan ukuran tiap kotak berdasarkan ukuran window. Kecepatan frame per detik (FPS) juga diatur menjadi 24, untuk memberikan pengalaman bermain yang lancar.

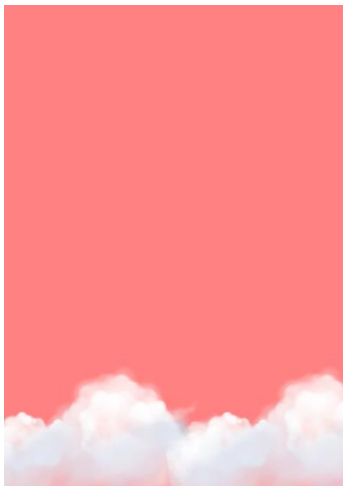
Tampilan window game akan diberi judul "Sudoku by IRAMA 🎮", yang akan terlihat di bagian atas jendela permainan. Selain itu, berbagai variabel game juga diinisialisasi, seperti font untuk teks dalam permainan, ukuran tombol, timer untuk menghitung durasi permainan, skor permainan, dan variabel lainnya yang mendukung jalannya permainan.

Pada tahap ini, berbagai asset grafis dan audio juga dimuat ke dalam game, yakni:

- Gambar latar belakang (background image)
- Gambar trofi untuk menandai kemenangan
- Tombol-tombol UI seperti tombol untuk memulai permainan baru, keluar, memilih tingkat kesulitan, kembali ke main menu, dan bermain kembali.
- Efek suara untuk berbagai aksi dalam permainan, seperti suara klik tombol, notifikasi khusus, *sound effect* jika user benar/salah input angka, dan musik jika user memenangkan permainan.
- Musik latar belakang untuk memberikan suasana bermain yang lebih immersive.
- Cursor khusus berupa gambar peri untuk menggantikan cursor default.

4. Metode-Metode Utama

a. `draw_background()`

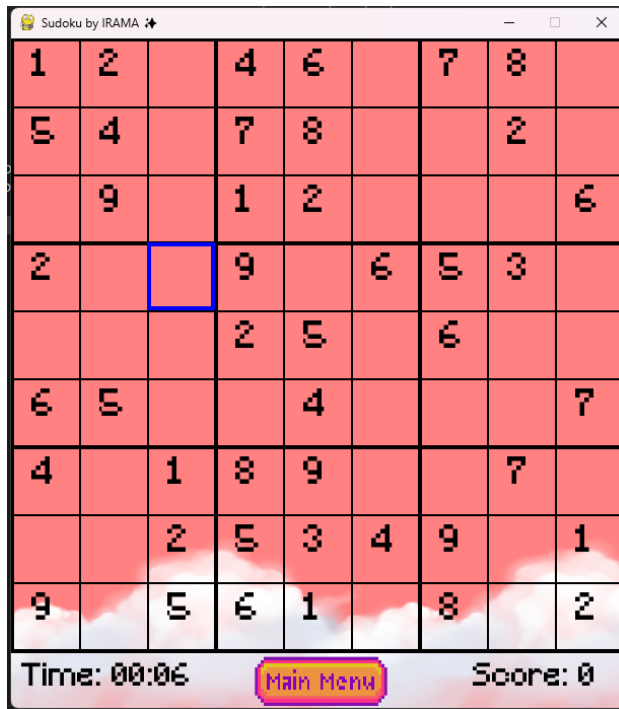


Metode ini bertanggung jawab untuk menggambar latar belakang permainan. Jika gambar latar belakang tersedia, maka gambar tersebut akan digambar pada window permainan. Jika tidak ada, maka metode ini akan mengisi latar belakang dengan warna putih. Hal ini penting untuk memberikan tampilan visual yang bersih dan menarik selama permainan.

b. `format_time(seconds)`

Metode ini berfungsi untuk mengonversi durasi waktu dalam detik menjadi format waktu yang lebih mudah dibaca, yaitu menit:detik (MM:SS). Misalnya, jika permainan sudah berlangsung selama 90 detik, maka metode ini akan mengubahnya menjadi "01:30". Fungsi ini akan digunakan untuk menampilkan waktu yang telah berlalu dalam permainan, memberikan pengguna gambaran tentang seberapa cepat mereka menyelesaikan puzzle.

c. `calculate_score(input_type)`



Metode ini digunakan untuk menghitung skor pemain berdasarkan jawaban yang mereka pilih. Setiap kali pemain memilih angka yang sesuai, mereka akan mendapatkan 10 poin. Namun, jika jawaban yang dipilih salah, mereka akan kehilangan 8 poin dan juga setiap 45 detik sekali akan ada *sound effect* khusus untuk mengurangi poin sebanyak 4 agar meningkatkan adrenalin pemain. Sistem skor ini dirancang untuk memberi tantangan dan memberikan insentif bagi pemain untuk berpikir dengan hati-hati sebelum mengisi.

d. `draw_grid()`

Metode ini bertugas untuk menggambar grid Sudoku pada window permainan. Grid ini terdiri dari 9x9 kotak yang membentuk teka-teki. Metode ini juga bertanggung jawab untuk menampilkan angka-angka yang sudah ada di dalam puzzle. Selain itu, kotak yang sedang dipilih oleh pemain akan ditandai dengan highlight berwarna biru agar pemain tahu di mana mereka sedang mengerjakan teka-teki. Di bagian bawah grid, metode ini juga menampilkan informasi penting seperti waktu yang telah berlalu, skor pemain, dan tombol untuk kembali ke menu utama.

e. `generate_puzzle(level)`

Metode `generate_puzzle` digunakan untuk menghasilkan teka-teki Sudoku dengan tingkat kesulitan yang berbeda. Berdasarkan tingkat kesulitan yang dipilih (Easy, Medium, Hard), puzzle akan dihasilkan dengan jumlah angka yang sudah terisi di dalam grid. Untuk level **Easy** dua angka akan dihapus dari grid (ini dibuat dua angka sebab untuk kebutuhan demonstrasi saja, untuk nilai asli adalah 27), kemudian untuk level **Medium** 37 angka akan dihapus dan untuk level **Hard**, 47 angka akan dihapus. Angka-angka yang tersisa di grid akan membentuk puzzle yang dapat diselesaikan oleh pemain. Angka-angka yang ada pada puzzle ini diacak secara acak dan diuji agar puzzle yang dihasilkan tetap memiliki satu solusi yang valid.

f. `solve_sudoku(board)`

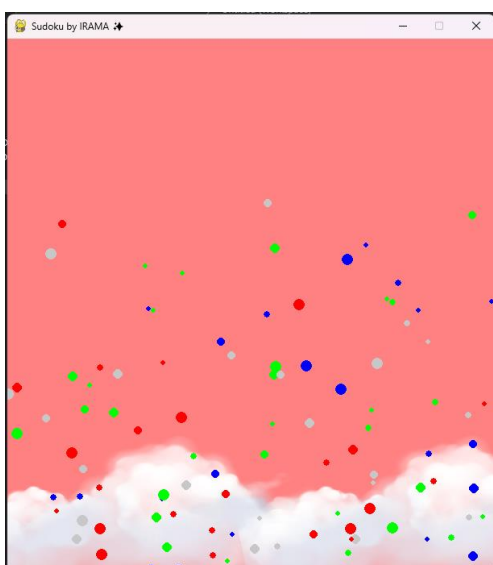
Metode `solve_sudoku` mengimplementasikan algoritma `backtracking` untuk menyelesaikan puzzle Sudoku. Algoritma ini berfungsi untuk mencari solusi dengan mencoba angka-angka satu per satu dan memeriksa apakah angka tersebut valid atau tidak. Validitas dicek dengan memastikan bahwa angka yang dimasukkan tidak bertentangan dengan aturan Sudoku (angka 1-9 hanya boleh muncul satu kali dalam setiap baris, kolom, dan subgrid). Jika suatu angka tidak valid, algoritma akan mundur dan mencoba angka lain di posisi yang sama, hingga seluruh puzzle terisi dengan benar.

Metode ini tidak hanya digunakan untuk menyelesaikan puzzle secara otomatis, tetapi juga membantu memeriksa validitas setiap angka yang dimasukkan oleh pemain selama permainan, memberikan umpan balik apakah jawaban mereka benar atau salah.

g. `draw_button(button_image, y_position, text)`

Metode `draw_button` bertanggung jawab untuk menggambar tombol-tombol antarmuka pengguna yang digunakan dalam game. Tombol ini dapat berupa gambar atau teks, tergantung pada parameter yang diberikan. Metode ini menerima tiga parameter utama: `button_image` yang menyimpan gambar tombol yang akan ditampilkan, `y_position` yang menunjukkan posisi vertikal tombol di dalam window, dan `text` yang berisi teks yang akan ditampilkan pada tombol tersebut misalnya, "New Game", "Quit", dll. Tombol akan digambar sesuai dengan posisi dan gambar yang diberikan, serta teks yang disesuaikan agar mudah dibaca oleh pemain.

h. `animate_confetti()`



Metode `animate_confetti` bertugas untuk membuat animasi konfeti yang muncul ketika pemain berhasil menyelesaikan puzzle dengan benar. Animasi ini digunakan untuk menambah kesan kegembiraan dan prestasi setelah menyelesaikan tantangan Sudoku. Konfeti yang jatuh secara acak di layar menciptakan efek visual yang menyenangkan dan merayakan kemenangan pemain. Elemen ini meningkatkan pengalaman bermain dengan memberikan umpan balik visual yang menyenangkan setelah pencapaian.

i. `main_menu()`

Metode `main_menu` digunakan untuk menampilkan layar menu utama permainan. Pada menu ini, pemain diberikan dua pilihan utama: tombol untuk memulai permainan baru "New Game" dan tombol untuk keluar dari permainan "Quit". Menu ini berfungsi sebagai titik awal bagi pemain untuk memulai permainan atau keluar dari aplikasi. Menu utama adalah bagian yang pertama kali ditampilkan setelah game dimulai dan memberikan opsi bagi pemain untuk memulai permainan atau mengakhiri sesi permainan.

j. `choose_difficulty()`



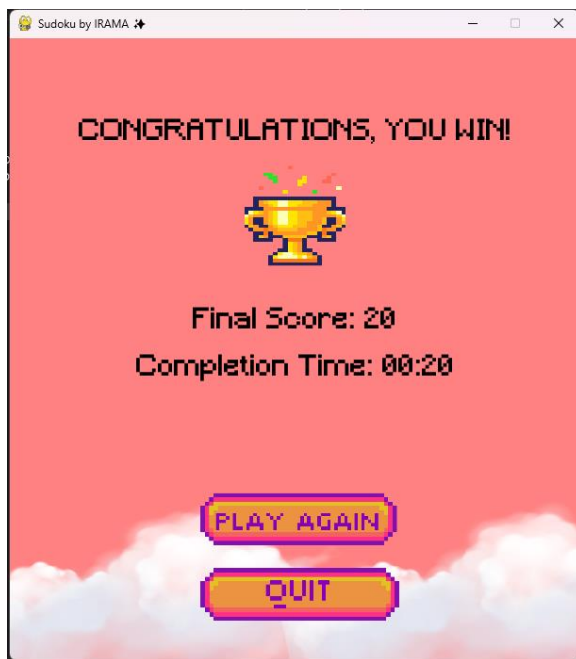
Metode `choose_difficulty` menampilkan layar pemilihan tingkat kesulitan setelah pemain memilih untuk memulai permainan baru. Pada layar ini, pemain dapat memilih salah satu dari tiga tingkat kesulitan: **Easy**, **Medium**, atau **Hard**. Pemain dapat memilih kesulitan yang sesuai dengan keinginan mereka, dan berdasarkan pilihan ini, jumlah angka yang dihapus pada puzzle akan bervariasi, memberikan tantangan yang berbeda-beda. Metode ini memberikan kontrol kepada pemain untuk menyesuaikan pengalaman bermain

mereka, baik bagi pemula yang baru belajar Sudoku atau pemain berpengalaman yang ingin tantangan lebih besar.

k. `is_game_won(board, solution)`

Metode `is_game_won` digunakan untuk mengecek apakah puzzle sudah selesai dengan benar atau belum. Metode ini membandingkan solusi yang diberikan oleh pemain dengan solusi yang benar (yang sudah dihitung dan disimpan dalam variabel `solution`). Jika seluruh angka yang dimasukkan pemain cocok dengan solusi yang benar, maka metode ini akan mengembalikan nilai `true`, menandakan bahwa permainan telah selesai dengan benar. Jika ada angka yang salah, maka permainan akan terus berlanjut. Fungsi ini sangat penting untuk memastikan bahwa pemain tidak hanya mengisi grid, tetapi juga mengisi dengan benar sesuai aturan Sudoku.

l. show_win_screen()



Metode `show_win_screen` menampilkan layar kemenangan ketika pemain berhasil menyelesaikan puzzle dengan benar. Layar ini menampilkan beberapa elemen penting:

- Animasi confetti, yang memberikan efek visual yang menyenangkan untuk merayakan kemenangan.
- Gambar trofi, yang menandakan bahwa pemain berhasil menyelesaikan permainan.
- Skor akhir pemain yang dihitung berdasarkan jawaban yang benar dan salah selama permainan.

- Waktu penyelesaian yang menunjukkan berapa lama waktu yang dihabiskan oleh pemain untuk menyelesaikan puzzle.
- Tombol Play Again yang memungkinkan pemain untuk memulai permainan baru tanpa keluar dari game, serta tombol Quit untuk keluar dari permainan.

Semua elemen ini dirancang untuk memberikan penghargaan kepada pemain setelah mereka menyelesaikan tantangan dengan sukses.

m. run_game()

Metode `run_game` adalah loop utama permainan yang menangani seluruh proses interaksi dan pembaruan dalam permainan. Beberapa hal yang dilakukan oleh metode ini meliputi:

- Mendeteksi input dari pemain baik menggunakan mouse (untuk menekan tombol) atau keyboard (untuk memilih angka).
- Memperbarui skor pemain berdasarkan input yang diberikan (jawaban yang benar atau salah).
- Setiap 45 detik, akan mengurangi skor pemain sebagai bentuk tantangan waktu, memberikan dorongan bagi pemain untuk bermain lebih cepat.
- Menggunakan metode `is_game_won` untuk memeriksa apakah permainan telah selesai dengan benar.

- Memutar efek suara untuk berbagai kejadian dalam permainan, seperti suara ketika pemain memasukkan angka atau ketika ada interaksi dengan tombol UI.

Metode ini akan terus berjalan selama permainan berlangsung, memastikan bahwa semua elemen dalam permainan terus diperbarui dan pemain dapat berinteraksi dengan permainan dengan lancar.

n. `start()`

Metode `start` adalah titik awal dari permainan, yang menangani seluruh proses mulai dari membuka game hingga memulai loop permainan. Di dalam metode ini, beberapa hal penting dilakukan:

- Menangani error yang mungkin terjadi selama permainan, seperti kesalahan dalam memuat file atau masalah teknis lainnya.
- Menginisialisasi dan memulai loop permainan dengan memanggil metode `run_game`.
- Mengatur seluruh jalannya permainan dari awal hingga akhir, serta memastikan bahwa seluruh aspek permainan berfungsi dengan baik.

Metode ini bertugas untuk memastikan permainan berjalan dengan lancar dan mengelola semua pengaturan awal yang diperlukan untuk memulai sesi permainan.

5. Fitur-Fitur Game Sudoku By IRAMA

1. Interface dan Tampilan:

- a) Menggunakan font Minecraft.ttf
- b) Cursor kustom berbentuk peri
- c) Grid Sudoku 9x9

2. Fitur Button Game:

- a) New Game : Memulai permainan baru
- b) Quit : Keluar dari game
- c) Play Again : Kembali ke menu utama
- d) Menu pilihan tingkat kesulitan:
 - *Easy*: 2 kotak kosong
 - *Medium*: 37 kotak kosong
 - *Hard*: 47 kotak kosong

3. Fitur *Gameplay*:

- a) Sistem Input:
 - Klik kiri mouse untuk memilih kotak
 - Tombol angka (1-9) untuk mengisi angka
 - Tombol anak panah atau WASD untuk navigasi antar kotak
- b) Validasi Puzzle:
 - Memberikan respon saat input benar/salah
 - Kotak yang salah diisi akan ditandai dengan warna merah
 - Kotak yang dipilih ditandai dengan warna biru

4. Sistem Scoring:

- a) Skor bertambah 10 poin untuk setiap input yang benar
- b) Skor berkurang 8 poin untuk setiap input yang salah
- c) Berkurang 4 poin setiap 45 detik
- d) Skor minimum adalah 0 (tidak bisa negatif)

5. Sistem Timer:

- a) Menampilkan waktu permainan dalam format MM:SS
- b) Timer mulai berjalan saat game dimulai
- c) Waktu penyelesaian ditampilkan di layar kemenangan

6. Audio System:

- a) Backsound
- b) Efek suara saat:
 - klik kiri mouse
 - Input benar
 - Input salah
 - Notifikasi pengurangan skor
 - Suara kemenangan

7. Generator Puzzle:

- a) Generator puzzle Sudoku dengan library random
- b) Tiga tingkat kesulitan berbeda:
 - *Easy*: 2 kotak yang dihapus
 - *Medium*: 37 kotak yang dihapus
 - *Hard*: 47 kotak yang dihapus

8. Fitur Cek Menang:

- a) Deteksi otomatis saat puzzle selesai
- b) Animasi konfeti saat selesai permainan
- c) Menampilkan skor akhir
- d) Menampilkan waktu penyelesaian
- e) Tombol kembali ke Menu utama

LAMPIRAN KODE UTAMA

```
import pygame

from pygame import mixer

import time

import random

import os


class SudokuGame:

    def __init__(self):

        pygame.init()

        mixer.init()


        self.WHITE = (225, 255, 225)

        self.BLACK = (0, 0, 0)

        self.RED = (255, 0, 0)

        self.green = (0, 255, 0)

        self.BLUE = (0, 0, 255)

        self.GREY = (200, 200, 200)

        self.WIDTH = 605

        self.HEIGHT = 660

        self.SQUARE_SIZE = self.WIDTH // 9

        self.FPS = 24
```

```

self.screen = pygame.display.set_mode((self.WIDTH, self.HEIGHT))

pygame.display.set_caption("Sudoku by IRAMA ✨")


base_dir = os.path.dirname(__file__)

assets_dir = os.path.join(base_dir, 'assets')

sounds_dir = os.path.join(base_dir, 'sounds')

self.font = pygame.font.Font(os.path.join(os.path.dirname(__file__),
"Minecraft.ttf"), 30)

self.font_2 =
pygame.font.Font(os.path.join(os.path.dirname(__file__), "Minecraft.ttf"), 36)

self.button_width = 360

self.button_height = 60

self.button_width2 = 600

self.button_height2 = 100

self.start_time = 0

self.elapsed_time = 0

self.score = 0

self.board = None

self.solution = None

self.selected = None

self.wrong_move = False

self.difficulty = None

self.answered_cells = set()

```

```

try:

    self.background = pygame.image.load(os.path.join(assets_dir,
'background.png'))

    self.background = pygame.transform.scale(self.background, (self.WIDTH,
self.HEIGHT))

except:

    print("Background is not found")


try:

    self.trophy = pygame.image.load(os.path.join(assets_dir, 'trophy.png'))

    self.trophy = pygame.transform.scale(self.trophy, (500, 500))

except:

    print("Trophy asset is not found")


try:

    self.new_game_btn = pygame.image.load(os.path.join(assets_dir, 'new
game.png'))

    self.quit_btn = pygame.image.load(os.path.join(assets_dir, 'quit.png'))

    self.easy_btn = pygame.image.load(os.path.join(assets_dir, 'easy.png'))

    self.medium_btn = pygame.image.load(os.path.join(assets_dir, 'medium.png'))

    self.hard_btn = pygame.image.load(os.path.join(assets_dir, 'hard.png'))

    self.play_again_btn = pygame.image.load(os.path.join(assets_dir, 'play
again.png'))

    self.back_menu_btn = pygame.image.load(os.path.join(assets_dir, 'back to
menu.png'))

```

```

        self.new_game_btn = pygame.transform.scale(self.new_game_btn,
        (self.button_width, self.button_height))

        self.quit_btn = pygame.transform.scale(self.quit_btn, (self.button_width,
        self.button_height))

        self.easy_btn = pygame.transform.scale(self.easy_btn, (self.button_width,
        self.button_height))

        self.medium_btn = pygame.transform.scale(self.medium_btn,
        (self.button_width, self.button_height))

        self.hard_btn = pygame.transform.scale(self.hard_btn, (self.button_width,
        self.button_height))

        self.play_again_btn = pygame.transform.scale(self.play_again_btn,
        (self.button_width, self.button_height))

        self.back_menu_btn = pygame.transform.scale(self.back_menu_btn, (300, 60))

    except:

        print("Button assets is not found")

    try:

        self.sudoku_game = pygame.image.load(os.path.join(assets_dir, 'sudoku
        game.png'))

        self.select_difficulty = pygame.image.load(os.path.join(assets_dir, 'select
        difficulty.png'))

        self.sudoku_game = pygame.transform.scale(self.sudoku_game,
        (self.button_width2, self.button_height2))

        self.select_difficulty = pygame.transform.scale(self.select_difficulty,
        (self.button_width2, self.button_height2))

    except:

```

```

print("Title assets is not found")

try:

    self.correct_sound = mixer.Sound(os.path.join(sounds_dir, 'correct.wav'))
    self.wrong_sound = mixer.Sound(os.path.join(sounds_dir, 'Wrong.mp3'))
    self.win_sound = mixer.Sound(os.path.join(sounds_dir, 'win.mp3'))
    self.button_sound = mixer.Sound(os.path.join(sounds_dir, 'button click.mp3'))
    self.notification_score = mixer.Sound(os.path.join(sounds_dir, 'notification
score.mp3'))

    pygame.mixer.music.load(os.path.join(sounds_dir, 'game sound.mp3'))
    pygame.mixer.music.set_volume(0.3)

except:

    print("Sounds is not found")

try:

    pygame.mixer.music.play(-1)

except:

    print("Music background is not found")

try:

    self.fairy_cursor = pygame.image.load(os.path.join(assets_dir, 'peri.png'))
    self.fairy_cursor = pygame.transform.scale(self.fairy_cursor, (32, 32))
    self.fairy_cursor_data = pygame.cursors.Cursor((16, 16), self.fairy_cursor)

except:

```

```

    print("Fairy cursor is not found")

    try:

        self.mouse_click_sound = mixer.Sound(os.path.join(sounds_dir, 'button
click.mp3'))

    except:

        print("Mouse click sound is not found")


def draw_background(self):

    if self.background:

        self.screen.blit(self.background, (0, 0))

    else:

        self.screen.fill(self.WHITE)


def format_time(self,seconds):

    minutes = seconds // 60

    seconds = seconds % 60

    return f"{int(minutes):02}:{int(seconds):02}"


def calculate_score(self,input_type):

    global score

    if input_type == 'correct':

        self.score += 10

    elif input_type == 'incorrect':

```

```

        self.score = max(0, self.score - 8)

    return self.score

def draw_grid(self):
    self.draw_background()

    for i in range(10):
        line_width = 4 if i % 3 == 0 else 2

        pygame.draw.line(self.screen, self.BLACK,
                          (i * self.SQUARE_SIZE, 0),
                          (i * self.SQUARE_SIZE, self.WIDTH), line_width)

        pygame.draw.line(self.screen, self.BLACK,
                          (0, i * self.SQUARE_SIZE),
                          (self.WIDTH, i * self.SQUARE_SIZE), line_width)

    for i in range(9):
        for j in range(9):
            if self.board[i][j] != 0:
                text = self.font_2.render(str(self.board[i][j]), True, self.BLACK)

                self.screen.blit(text, (j * self.SQUARE_SIZE + 20, i * self.SQUARE_SIZE +
10))

    if self.selected:
        row, col = self.selected

        color = self.RED if self.wrong_move else self.BLUE

```

```

pygame.draw.rect(self.screen, color,

                    (col self.SQUARE_SIZE, row self.SQUARE_SIZE,
self.SQUARE_SIZE, self.SQUARE_SIZE), 4)

elapsed_seconds = time.time() - self.start_time

stopwatch_text = self.font.render("Time: " + self.format_time(elapsed_seconds),
True, self.BLACK)

self.screen.blit(stopwatch_text, (10, self.WIDTH + 10))

self.score_text = self.font.render(f"Score: {self.score}", True, self.BLACK)

self.screen.blit(self.score_text, (self.WIDTH - 150, self.WIDTH + 10))

if self.back_menu_btn:

    back_btn_x = (self.WIDTH - 294) // 2

    back_btn_y = self.WIDTH - 2

    back_menu_rect = self.back_menu_btn.get_rect(topleft=(back_btn_x,
back_btn_y))

    self.screen.blit(self.back_menu_btn, back_menu_rect)

    return back_menu_rect

def generate_puzzle(self, level):

    def shuffle_numbers():

        numbers = list(range(1, 10))

        random.shuffle(numbers)

```

```
    return numbers

    self.board = [[0 for _ in range(9)] for _ in range(9)]

    numbers = shuffle_numbers()

    for i in range(9):

        self.board[i][i] = numbers[i]

    self.solve_sudoku(self.board)

    self.solution = [row[:] for row in self.board]

    if level == 'easy':

        num_remove = 2

    elif level == 'medium':

        num_remove = 37

    else:

        num_remove = 47

    removed_positions = set()

    while len(removed_positions) < num_remove:

        row = random.randint(0, 8)

        col = random.randint(0, 8)

        if (row, col) not in removed_positions:

            removed_positions.add((row, col))

            self.board[row][col] = 0
```

```
return self.board, self.solution
```

```
def solve_sudoku(self, board):
```

```
    def is_valid(pos, num):
```

```
        row, col = pos
```

```
        if num in board[row]:
```

```
            return False
```

```
        for i in range(9):
```

```
            if board[i][col] == num:
```

```
                return False
```

```
        box_x, box_y = (row // 3) * 3, (col // 3) * 3
```

```
        for i in range(box_x, box_x + 3):
```

```
            for j in range(box_y, box_y + 3):
```

```
                if board[i][j] == num:
```

```
                    return False
```

```
        return True
```

```
def find_empty():
```

```
    for i, row in enumerate(board):
```

```
        for j, val in enumerate(row):
```

```
            if val == 0:
```

```
                return (i, j)
```

```
    return None
```

```

empty = find_empty()

if not empty:

    return True


row, col = empty

for num in range(1, 10):

    if is_valid((row, col), num):

        board[row][col] = num

        if self.solve_sudoku(board):

            return True

        board[row][col] = 0


return False


def draw_button(self, button_image, y_position, text=None):

    if button_image is None:

        color = self.GREY

        button_width, button_height = 300, 50

        button_x = (self.WIDTH - button_width) // 2

        button = pygame.Rect(button_x, y_position, button_width, button_height)

        pygame.draw.rect(self.screen, color, button, border_radius=20)

        if text:

```

```

        button_text = self.font.render(text, True, self.BLACK)

        text_rect = button_text.get_rect(center=button.center)

        self.screen.blit(button_text, text_rect)

    else:

        button_x = (self.WIDTH // 4.75)

        button = pygame.Rect(button_x, y_position, self.button_width,
self.button_height)

        self.screen.blit(button_image, button)

        return button

def play_button_sound(self):

    if self.button_sound:

        self.button_sound.play()

def animate_confetti(self):

    particles = []

    for _ in range(200):

        x = random.randint(0, self.WIDTH)

        y = random.randint(0, self.HEIGHT)

        color = random.choice([self.RED, self.BLUE, self.GREY, self.green])

        size = random.randint(3, 7)

        speed = random.uniform(1, 5)

        particles.append([x, y, size, speed, color])

```

```

for _ in range(380):

    self.screen.blit(self.background, (0, 0))

    for p in particles:

        p[1] += p[3]

        pygame.draw.circle(self.screen, p[4], (p[0], int(p[1])), p[2])

    pygame.display.flip()

    pygame.time.delay(7)

def main_menu(self):

    self.screen.blit(self.background, (0, 0))

    if self.sudoku_game:

        title_rect = self.sudoku_game.get_rect(center=(self.WIDTH//2, 100))

        self.screen.blit(self.sudoku_game, title_rect)

    start_button = self.draw_button(self.new_game_btn, 250)

    quit_button = self.draw_button(self.quit_btn, 350)

    pygame.display.update()

while True:

    for event in pygame.event.get():

        if event.type == pygame.QUIT:

            return 'quit'

        if event.type == pygame.MOUSEBUTTONDOWN:

```

```

        mouse_pos = pygame.mouse.get_pos()

        if start_button.collidepoint(mouse_pos):

            if self.button_sound:

                self.button_sound.play()

            return 'start'

        elif quit_button.collidepoint(mouse_pos):

            if self.button_sound:

                self.button_sound.play()

            return pygame.quit()

def choose_difficulty(self):

    self.screen.blit(self.background, (0, 0))

    if self.select_difficulty:

        title_rect = self.select_difficulty.get_rect(center=(self.WIDTH//2, 100))

        self.screen.blit(self.select_difficulty, title_rect)

    easy_button = self.draw_button(self.easy_btn, 250)

    medium_button = self.draw_button(self.medium_btn, 350)

    hard_button = self.draw_button(self.hard_btn, 450)

    pygame.display.update()

    while True:

```

```

for event in pygame.event.get():

    if event.type == pygame.QUIT:

        pygame.quit()

        return None

    if event.type == pygame.MOUSEBUTTONDOWN:

        mouse_pos = pygame.mouse.get_pos()

        if easy_button.collidepoint(mouse_pos):

            if self.button_sound:

                self.button_sound.play()

            return 'easy'

        if medium_button.collidepoint(mouse_pos):

            if self.button_sound:

                self.button_sound.play()

            return 'medium'

        if hard_button.collidepoint(mouse_pos):

            if self.button_sound:

                self.button_sound.play()

            return 'hard'

def is_game_won(self, board, solution):

    return all(board[i][j] == solution[i][j] for i in range(9) for j in range(9))

def show_win_screen(self):

```

```

pygame.mixer.music.stop()

if self.win_sound:

    self.win_sound.play()

self.animate_confetti()

self.screen.blit(self.background, (0, 0))


if self.trophy:

    trophy_rect = self.trophy.get_rect(centerx=self.WIDTH // 2, top=40)

    self.screen.blit(self.trophy, trophy_rect)


texts = [

    ("CONGRATULATIONS, YOU WIN!", 100),

    (f'Final Score: {self.score}', 300),

    (f'Completion Time: {self.format_time(time.time() - self.start_time)}', 350)]

for text, y in texts:

    surface = self.font.render(text, True, self.BLACK)

    rect = surface.get_rect(center=(self.WIDTH//2, y))

    self.screen.blit(surface, rect)


buttons = {

    'play_again': (self.play_again_btn, self.HEIGHT // 2 + 150),

    'quit': (self.quit_btn, self.HEIGHT // 2 + 230)}

```

```

button_rects = {

    name: self.draw_button(img, y_pos)

    for name, (img, y_pos) in buttons.items()}

pygame.display.update()

while True:

    for event in pygame.event.get():

        if event.type == pygame.QUIT:

            pygame.quit()

            exit()

        if event.type == pygame.MOUSEBUTTONDOWN:

            mouse_pos = event.pos

            if button_rects['play_again'].collidepoint(mouse_pos):

                self.play_button_sound()

                try:

                    pygame.mixer.music.play(-1)

                except:

                    print("Music background is not found")

            return

        elif button_rects['quit'].collidepoint(mouse_pos):

```

```
        self.play_button_sound()

        pygame.quit()

        exit()

def run_game(self):

    clock = pygame.time.Clock()

    running = True

    pygame.mouse.set_cursor(self.fairy_cursor_data)

    while running:

        menu_choice = self.main_menu()

        if menu_choice == 'quit':

            pygame.quit()

            return

        level = self.choose_difficulty()

        if level is None:

            pygame.quit()

            return

        self.board, self.solution = self.generate_puzzle(level)

        self.start_time = time.time()

        self.score = 0
```

```

self.selected = (0, 0)

self.wrong_move = False

self.answered_cells = set()

game_running = True

last_score_reduction = time.time()

score_reduction_interval = 45

score_reduction_amount = 4


while game_running:

    clock.tick(self.FPS)

    current_time = time.time()

    if current_time - last_score_reduction >= score_reduction_interval:

        self.score = max(0, self.score - score_reduction_amount)

        last_score_reduction = current_time

        if self.score:

            self.notification_score.play()


    for event in pygame.event.get():

        if event.type == pygame.QUIT:

            running = False

            game_running = False

            return

```

```

if event.type == pygame.MOUSEBUTTONDOWN:

    pos = pygame.mouse.get_pos()

    back_menu_rect = self.draw_grid()

    if back_menu_rect.collidepoint(pos):

        if self.button_sound:

            self.button_sound.play()

            game_running = False

            break

    row, col = pos[1] // self.SQUARE_SIZE, pos[0] // self.SQUARE_SIZE

    if row < 9 and col < 9:

        self.selected = (row, col)

    if event.type == pygame.USEREVENT + 1 or event.type ==
pygame.USEREVENT + 2:

        self.restore_background_volume()

    if event.type == pygame.KEYDOWN:

        if event.key in [pygame.K_LEFT, pygame.K_a] and self.selected[1] > 0:

            self.selected = (self.selected[0], self.selected[1] - 1)

        elif event.key in [pygame.K_RIGHT, pygame.K_d] and self.selected[1] <
8:

            self.selected = (self.selected[0], self.selected[1] + 1)

        elif event.key in [pygame.K_UP, pygame.K_w] and self.selected[0] > 0:

```

8:

```
        self.selected = (self.selected[0] - 1, self.selected[1])

    elif event.key in [pygame.K_DOWN, pygame.K_s] and self.selected[0] < 8:

        self.selected = (self.selected[0] + 1, self.selected[1])

    if pygame.K_1 <= event.key <= pygame.K_9:

        num = event.key - pygame.K_0

    elif pygame.K_KP1 <= event.key <= pygame.K_KP9:

        num = event.key - pygame.K_KP1 + 1

    else:

        num = None

    if self.selected and num:

        row, col = self.selected

        if (row, col) not in self.answered_cells and self.board[row][col] == 0:

            if self.solution[row][col] == num:

                self.board[row][col] = num

                self.correct_sound.play()

                self.wrong_move = False

                self.score = self.calculate_score('correct')

            else:

                self.wrong_sound.play()

                self.wrong_move = True

                self.score = self.calculate_score('incorrect')
```

```
        if game_running:

            back_menu_rect = self.draw_grid()

            pygame.display.flip()

        if self.is_game_won(self.board, self.solution):

            self.show_win_screen()

            game_running = False

    def start(self):

        try:

            self.run_game()

        except Exception as e:

            print(f'Error Occurred: {e}')

        finally:

            pygame.quit()

if __name__ == "__main__":

    game = SudokuGame()

    game.start()
```

DAFTAR PUSTAKA

- Hadinata, J., Tinggi, S., Informatika, M., & Pontianak, D. K. (n.d.). *Problem Solving Sudoku Menggunakan ... 48 Jurnal Ilmiah SISFOTENIKA PROBLEM SOLVING SUDOKU MENGGUNAKAN ALGORITMA GENETIKA*.
- Merwin, R., O'Rourke, S., Ives, L., Harstad, S., & Brooks, J. (2019). Third-wave cognitive-behavioral therapies for the treatment of anxiety among children and adolescents. In *Pediatric Anxiety Disorders* (pp. 335–357). Elsevier. <https://doi.org/10.1016/B978-0-12-813004-9.00016-5>
- Rahman, S., Sembiring, A., Siregar, D., Khair, H., Prahmana, G., Puspadini, R., & Zen, M. (n.d.). *PYTHON : DASAR DAN PEMROGRAMAN BERORIENTASI OBJEK TAHTA MEDIA GROUP*.